Système d'exploitation : TD1

janvier 2026 — Pierre Rousselin

Exercice 1 : Mousse au chocolat

Voici une recette de mousse au chocolat.

Ingrédients et matériel

- une tablette de chocolat à dessert
- 6 œufs
- une pincée de sel
- un saladier pour la mousse
- un récipient pour monter les blancs en neige
- un batteur électrique sinon c'est fatigant
- une casserolle pour faire fondre le chocolat (de préférence au bain-marie)

Recette très détaillée

1. Casser la tablette de chocolat en carrés à mettre dans la casserolle. (durée : 2)
2. Faire fondre le chocolat. (durée : 10)
3. Pour chaque œuf, séparer le blanc du jaune, mettre le blanc dans le récipient et le jaune dans le saladier. (durée : 2 pour chaque œuf)
4. Mélanger les jaunes d'œuf avec le chocolat fondu. (durée : 2)
5. Ajouter une pincée de sel dans le récipient et utiliser le batteur pour monter les blancs en neige. (durée : 6)
6. Incorporer délicatement les blancs en neige dans le chocolat. (durée : 10)

1. Combien de temps prend le processus « mousse au chocolat » pour un seul cuisinier ?

2. Il y a 6 tâches, et même 11 si l'on compte une tâche par œuf à séparer en blanc et jaune. Quelles sont les dépendances entre les tâches (c'est-à-dire, pour chaque tâche, quelles sont les tâches qui doivent être terminées avant de pouvoir la commencer) ? Dessiner un diagramme de dépendance.

3. Chercher des moyens, avec autant de cuisiniers que vous voulez de rendre la recette parallèle, de sorte qu'elle prenne le moins de temps possible (on a faim !).

--- * ---

Voici quelques rappels sur les fonctions `fork`, `getpid` et `getppid`.

```
#include <unistd> /* fork */
#include <sys/types.h> /* type pid_t */

/* Crée un nouveau processus en dupliquant le processus appelant. Retourne :
 * -1 en cas d'échec,
 * 0 pour le processus enfant,
 * un entier strictement positif pour le processus parent qui est le PID
 * (identificateur de processus) du processus enfant nouvellement créé. */
pid_t fork(void);
/* retourne le PID du processus appelant */
pid_t getpid(void);
/* retourne le PPID, c'est-à-dire le PID du parent du processus appelant */
pid_t getppid(void);
```

Exercice 2 :

On considère le programme suivant.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

void aff_proc(const char *ch);

int main(void)
{
        switch (fork()) {
        case -1:
                perror("fork 1");
                return 1;
        case 0:
                aff_proc("Enfant 1");
                break;
        default:
                aff_proc("Parent 1");
        }

        switch (fork()) {
        case -1:
                perror("fork 2");
                return 1;
        case 0:
                aff_proc("Enfant 2");
                break;
        default:
                aff_proc("Parent 2");
        }

        printf("Je suis le processus %d et j'ai terminé\n", getpid());
        return 0;
}

void aff_proc(const char *ch)
{
        printf("%s : PID : %d, PPID : %d\n", ch, getpid(), getppid());
}
```

1. Combien de processus sont créés ?
2. Donner le nombre d'occurrences de chaque chaîne `(Parent|Enfant) [12]`, sur la sortie du programme.
3. Donner l'arbre de création des processus.
4. Est-ce que l'ordre dans lequel les processus se terminent est fixé ?
5. Voici un rappel sur la fonction `wait`.

   ```c
   #include <sys/types.h>
   #include <sys/wait.h>
   /* attend la terminaison d'un des processus enfants
    * retourne le PID de l'enfant qui a terminé (-1 en cas d'erreur, par exemple
    * s'il n'y a plus d'enfant à attendre), et met son statut de sortie à
    * l'adresse statut ou l'ignore si elle vaut NULL */
   pid_t wait(int *statut);
   ```

   Modifier le programme précédent de sorte que chaque parent attende la terminaison de tous ses enfants avant de se terminer lui-même. Chaque parent devra afficher, chaque fois qu'un de ses enfants se termine la chaîne suivante :

`Je suis <pid> et mon enfant de pid <pid> est mort.` en remplaçant les chaînes entre chevron par les valeurs appropriées.

--- * ---

Exercice 3 : Plein de `fork`

On considère le programme suivant :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#define NUM_ITER_DEFAUT 3
int main(int argc, char *argv[])
{
        int num_iter, i;
        num_iter = argc < 2 ? NUM_ITER_DEFAUT : atoi(argv[1]);
        for (i = 0; i < num_iter; ++i) {
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        printf("plop\n");
                        break;
                default:
                        switch (fork()) {
                        case -1:
                                perror("fork");
                                return 1;
                        case 0:
                                break;
                        default:
                                printf("hop\n");
                                return 0;
                        }
                }
        }
        printf("bloup\n");
        return 0;
}
```

1. Quelle est la valeur de `num_iter` si le programme est exécuté sans argument, c'est-à-dire ainsi :
   `$ ./plein_de_fork`
2. On suppose que le programme a été exécuté de la façon suivante :
   `$ ./plein_de_fork 2`
   Dessiner l'arbre de création des processus dans ce cas. Combien de fois sont écrites les chaînes `plop`, `hop` et `bloup` ?
3. En général, quel est, en fonction de l'argument donné par l'utilisateur, le nombre total de processus créés, le nombre de `plop`, `hop` et `bloup` qui sont affichés ?
4. (Bonus) Comment diable expliquer la chose suivante ?
   ```
   $ gcc -Wall -O0 plein_de_forks.c
   $ ./a.out 2 | grep plop | wc -l
   5
   ```

--- * ---